Architecture Justification – Hostel Maintenance & Issue Ticketing System

Name : P. Saanvi
SRN : PES1UG24AM905
Section : C
Team : 007

Architecture Selection: We chose Layered Architecture for the Hostel Maintenance & Issue Ticketing System.

Architectural Choice: Layered Architecture structured into seven components across three distinct layers. Presentation: Resident Portal and Warden & Staff Console. Business: Ticket Intake & Triage Service (FR-001, FR-005), Staff Dispatch Service (FR-002), Job Status Tracking Service (FR-003) and SLA & Closure Service (FR-004, NFR-001). Data: Data Access & Media Storage Component and Relational Database & Media Store. Each layer communicates strictly with adjacent layers through provided and required assembly interfaces including ITicketSubmission, IIntakeDashboard, IDispatchControl, IStatusUpdate, ISignOffClosure, INewTicketFeed, IWorkOrder, IStatusEvent, IDataAccess and IStorage.

Reason 1 – Decoupling Resident Issue Intake from Operational Dispatch: Residents log maintenance tickets with dynamic room number tags, multi-category classifications (plumbing, electrical, carpentry), and compressed photo uploads, which the warden oversees via an intake dashboard. In our design, all input validation, priority calculation, and dashboard feeds reside encapsulated inside the Ticket Intake & Triage Service. Modifying intake validation schemas, category definitions, or warden dashboard sorting criteria requires no adjustments to technician dispatch pipelines, status updates, closure sign-offs, or underlying database table schemas.

Reason 2 – Clear Team Modularization Suited for Campus Intranet Scale: The maintenance workflow comprises four sequential phases (intake, dispatch, technician progress, closure sign-off) developed by a team of four students. Decoupling the business tier into cohesive, interface-driven services allows each team member to independently construct and unit test their assigned module against explicit interface mocks without blocking parallel work. Adopting a Microservices architecture would introduce unnecessary operational complexity, inter-service network overhead, and distributed transaction boundaries unwarranted for a campus-scale workload, while a monolithic Client-Server model would tightly interlace presentation logic with database queries.

Security Advantage: The Data Access & Media Storage Component functions as the single secure gateway to persistence. Front-end client portals are architecturally prohibited from making direct database connections. Every business request must funnel through IDataAccess, ensuring role-based access control (preventing residents from tampering with dispatch states or staff work orders) and enforcing strict validation and AES-256 encryption on resident identity records and uploaded photographic attachments at rest.

Performance Benefit: Business services are completely stateless, permitting horizontal scale-out behind a reverse proxy to easily handle traffic spikes during semester intake surges. Additionally, the Ticket Intake & Triage Service maintains active, unassigned ticket summaries in memory, delivering sub-2-second sorting response times on the Warden Intake Dashboard (FR-005) without triggering expensive, repetitive relational table scans.

Trade-off: Enforcing strict layer isolation introduces minor call-overhead across boundaries, and individual services cannot be deployed or scaled in full infrastructure isolation compared to Microservices. We accept this trade-off for substantially cleaner development velocity, unambiguous component responsibilities, and robust, centralized data security.